



Module Code	:	CT065-3-3-ALG
Intake Code	:	APD3F2601CS(CYB)
Assignment	:	Group Assignment 1: Algorithmic Performance and Evaluation
Title	:	Evaluation of Disjoint Set Union (Union-Find) Using Find with Path Compression and Dynamic Connectivity
Lecturer Name	:	Maznah Binti Mohd Zin
Hand in Date	:	10 th May 2026

Student ID	Student Name
TP070844	Hemanthrajah A/L N Krishnarajah
TP071154	Anusha Devy A/P Brahmataran
TP070071	Darmik Ishwar A/L Ravi
TP070581	Srisudharson A/L Perumal

Table of Contents

1.0 Introduction.....	5
2.0 Selected Data Structure and Algorithm.....	6
2.1 Disjoint Set Union (Union-Find).....	6
2.2 Algorithm 1: Find with Path Comparison.....	8
2.3 Algorithm 2: Dynamic Connectivity using DSU	9
2.4 Important Design Limitation	10
3.0 Question 1: Understanding Polynomial-Time (P) Problems	11
3.1 Concept and Identification.....	11
3.2 Time Complexity Analysis	12
3.2.1 Operation 1: Availability Check.....	12
3.2.2 Operation 2: Dynamic Connectivity Update	12
3.3 Practical Implication for Real-Time Smart Library	14
4.0 Question 2: Understanding Non-Deterministic Polynomial-Time (NP) Problem.....	15
4.1 Concept and Identification.....	15
4.2 Complexity Analysis of Fully Optimal Recommendation	16
4.3 Why the NP approach becomes Inefficient	17
5.0 Question 3: P vs NP Comparison in Smart Library.....	18
5.1 Which Operation must Remain in P and Why	20
5.2 Which Operations can Tolerate Higher Complexity	20
6.0 Question 4: Decision Making and Optimization	21
6.1 Algorithm Selection	21
6.2 Real-World Optimization Technique: Precomputation and Caching with Greedy Ranking.....	21
6.3 Proposed Online Recommendation Algorithm	23
7.0 Conclusion	24
8.0 References.....	25
Appendix A: AI Tool Disclosure.....	26

List of Figures

Figure 1: DSU-based architecture for the Smart Library recommendation support layer.....	7
Figure 2: Pseudocode for Find with Path Compression.....	8
Figure 3: Pseudocode for Dynamic Connectivity update in Smart Library.....	9
Figure 4: Dynamic Connectivity flow for a borrowing event.....	10
Figure 5: P vs NP placement of Smart Library operations	18
Figure 6: Pseudocode for the recommended polynomial online recommendation pipeline.....	23

List of Tables

Table 1: Smart Library scale assumptions used for complexity analysis	6
Table 2: Selected Smart Library P operations	11
Table 3: Best-case and worst-case considerations for selected algorithms	13
Table 4: Exact recommendation compared with polynomial alternatives	17
Table 5: P vs NP comparison for the Smart Library case study	19
Table 6: Techniques to handle NP complexity in Smart Library recommendations	22

1.0 Introduction

SmartLib University is defined as a library system that should have capability to accommodate 100,000 books, 15,000 current students, perform nearly 2,000 searches per day, handle approximately 500 borrowings, return approximately 450 books every day, maintain a live popularity statistic, respond to nearly 300 requests for recommendations, etc. Also, the case study suggests that a maximum of 500 current users can access SmartLib at peak times (Asia Pacific University of Technology and Innovation, 2026). Thus, such numbers make the case study a good example for analyzing polynomial-time versus NP-style optimization problem. In other words, the problem in question gives us a chance to evaluate why polynomial-time problem can be resolved online, but NP optimization problem cannot be easily solved.

In this project report, the implementation of the chosen algorithm and data structure will be done through the application in the chosen case study, which include Disjoint set union (DSU), Find using path compression, and Dynamic connectivity. DSU algorithm will be chosen because it is used to maintain disjoint sets of elements where each new connection can be made between any two elements. In Smart Library, the books can be treated as nodes, where when there is a link shown through borrowings records of books being related to one another, like having been borrowed by the same user or in the same subject paths or recommendations, they can be merged together.

The paper does not argue that DSU alone would solve all the issues in libraries. Exact title searches, ISBN searches, and other similar operations are better implemented using hash indexing, inverted indexing, queue structures, and database tables. The real advantage of DSU lies in its use as a supporting data structure to implement the concept of dynamic similarity connectivity. This leads to a recommendation engine that is both fast and scalable in nature.

Case-study item	Value from brief	Design meaning
Total books	100,000	The DSU universe contains up to 100,000 book nodes.
Active students	15,000	Borrowing histories create many student-book and book-book relationships.
Daily searches	2,000	Search must use indexes and remain polynomial, preferably near constant/logarithmic.
Daily borrowings / returns	500 / 450	Popular-list, availability, and dynamic relationship updates happen continuously.
Recommendation requests	300 per day	Exact combinatorial optimization should not be performed online.
Average student history	30 books	Each new borrow can create about 30 candidate similarity unions.

Table 1: Smart Library scale assumptions used for complexity analysis

2.0 Selected Data Structure and Algorithm

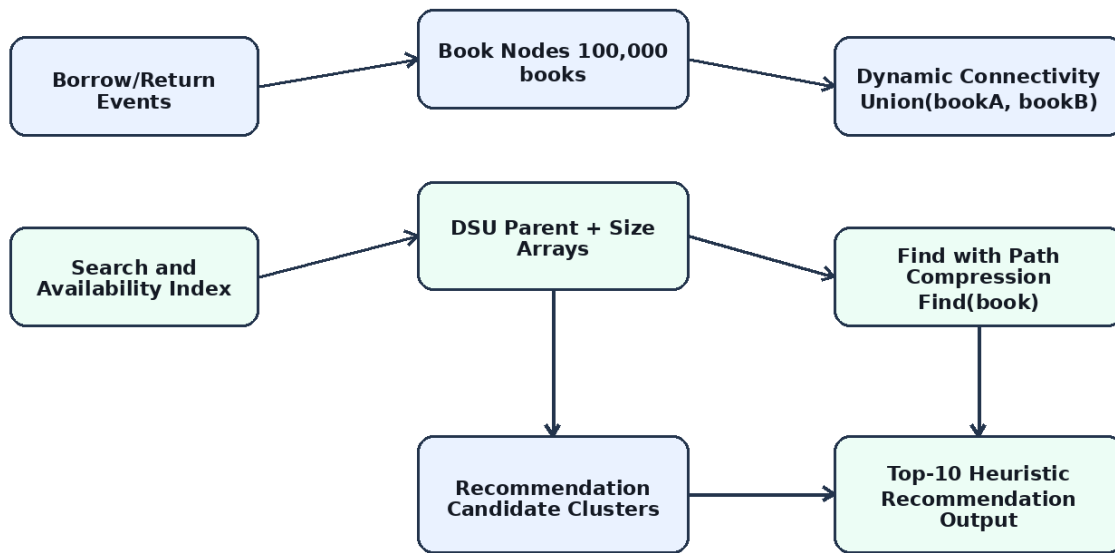
2.1 Disjoint Set Union (Union-Find)

Disjoint set union (Union-Find) keeps track of a group of disjointed sets. It allows three basic operations – initialization, Find, and Union. According to the documentation of the Linux kernel, Union-Find structure is a type of data structure which combines disjointed sets and provides a representative of these groups through Find function and unites them using the Union operation (The Linux Kernel documentation, 2024). In this case, DSU contains two arrays including a parent array and a size/rank array that contain information about books. If two books are similar, the system will join two sets in order to make further actions such as finding out if two elements belong to the same similarity group.

In the case of Smart Library DSU is used to implement the book-similarity layer dynamically. The basis for creating relationships can be borrowing statistics, book subjects, same authors, course book list, etc. For instance, when books A and B are constantly borrowed by students who study

the same module, the two books can be united in one component to speed up the candidate generation procedure, since there will not be necessary to browse 100,000 books.

Smart Library DSU-Based Architecture



Online path: update arrays, compress paths, and return cluster-based candidates within the response-time target.

Figure 1: DSU-based architecture for the Smart Library recommendation support layer

2.2 Algorithm 1: Find with Path Comparison

Find with Path Compression enhances the regular Find algorithm by ensuring that each accessed node directs itself towards the root node. In doing so, any subsequent Find searches conducted along the same path will run more quickly, since the tree structure becomes much shorter. Modern study of the Union-Find algorithm has demonstrated that its improved form could have an amortized running time of $O(\alpha(n))$, where $\alpha(n)$ refers to the inverse Ackermann function, which increases very slowly even with large values of n (Huang et al., 2023).

```

Pseudocode.txt
1  Find(bookID):
2      if parent[bookID] != bookID:
3          parent[bookID] = Find(parent[bookID])
4      return parent[bookID]
5
```

Figure 2: Pseudocode for Find with Path Compression

2.3 Algorithm 2: Dynamic Connectivity using DSU

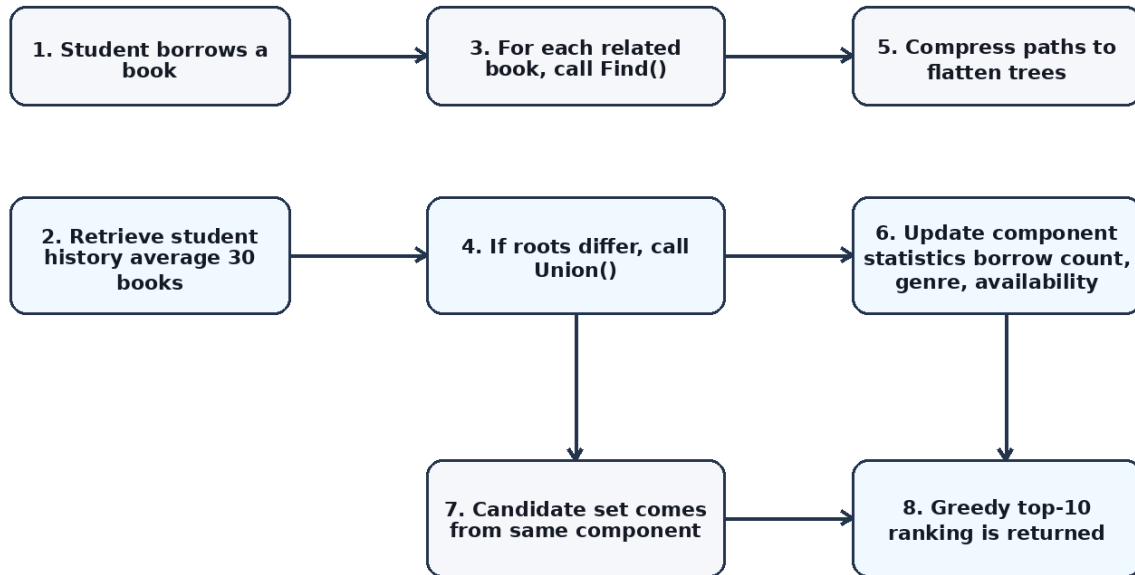
Dynamic connectivity determines whether two elements belong to the same connected component when edges are added. In this task, books are nodes, while the edge represents their relatedness level sufficient for grouping. Union-Find is usually employed to solve dynamic connectivity problems and issues in graphs because it allows incrementally updating the connected components without running a graph traversal algorithm each time the graph changes (The Linux Kernel documentation, 2024).

A borrowing operation may update the data structure in the following manner. Upon borrowing a book by a student, the system reads the borrowing history of this student. As it was mentioned earlier in the case study, on average, each student has a history of borrowing 30 books. Therefore, the system could compare the relatedness of a newly borrowed book with up to 30 books from the borrowing history. Whenever the relatedness level is sufficiently high, the system runs `Union(newBook, historyBook)`. Afterward, the recommendation module uses `Find(newBook)` to retrieve the component representative and rank books within the cluster.

```
6
7  UpdateConnectivity(studentID, newBook):
8      history = BorrowHistory[studentID]
9      for oldBook in history:
10         if Similarity(newBook, oldBook) >= threshold:
11             rootA = Find(newBook)
12             rootB = Find(oldBook)
13             if rootA != rootB:
14                 Union(rootA, rootB)          // attach smaller component to larger component
15             UpdateComponentStats(Find(newBook))
16
```

Figure 3: Pseudocode for Dynamic Connectivity update in Smart Library

Dynamic Connectivity Flow for Borrowing Event



The exact NP-style search is avoided during the online request. The system uses precomputed DSU components and a |

Figure 4: Dynamic Connectivity flow for a borrowing event

2.4 Important Design Limitation

An ordinary DSU is highly effective in terms of incremental connectivity, but it cannot be used for the deletion of the older relationship. This is important since Smart Library receives 450 returns every day. While returning a book will instantly update its availability and number of popular books, there is no need to instantly delete the relationship between the book similarities. In order to account for changes in preferences, we can simply regenerate the DSU offline using a certain window of time, such as daily or every few hours.

3.0 Question 1: Understanding Polynomial-Time (P) Problems

3.1 Concept and Identification

A Polynomial-Time problem is a problem that can be solved through an algorithm using a polynomial amount of resources like time or memory. The time complexity of solving such problems can range from $O(n)$ to $O(n^c)$ (Bun, 2023). In other words, the class P can be formally represented as the union of deterministic time classes $DTIME(n^c)$, where c is a constant. The class P often denotes the problems that can be solved fast enough in practical systems, particularly when the exponent c is small.

The two chosen operations from the Smart Library problem domain, which belong to the P class, are Availability Check and Dynamic Connectivity Update. The former is chosen since students need to have immediate knowledge about whether the book is already available or when it will be returned. The latter operation is chosen because it directly uses the DSU algorithm, Find operation, and Union operations to keep similarity components up-to-date.

Selected operation	P	Smart Library purpose	Data structure / algorithm	Reason it is P
Availability Check		Check whether a book can be borrowed now or when it is expected back.	Hash map by ISBN/bookID, availability table, optional return-date priority queue.	Exact lookup is $O(1)$ average; return-date retrieval is $O(\log r)$ if a queue is used, where r is active loans for a book.
Dynamic Connectivity Update		Merge related books into similarity components after borrow events.	DSU with Find Path Compression and Union by size/rank.	Each Find/Union is $O(\alpha(n))$ amortized, which is polynomial and practically near constant.

Table 2: Selected Smart Library P operations

3.2 Time Complexity Analysis

3.2.1 Operation 1: Availability Check

The availability function accepts an identifier for a book, typically its ISBN or bookID, and determines the status of that book, either the number on hand or the loan status of the copies. Given a hash table indexing based upon either ISBN or bookID, the average expected search time would be $O(1)$. If the library system needs to display the earliest return date from loaned copies, the loans themselves may be indexed using a min-priority queue sorted based on due dates, which allows constant-time $O(1)$ retrieval of the current status and a logarithmic $O(\log r)$ update when the status of a loan or return causes the change to occur, where r is the number of loan records in use for this particular book, rather than the total number of 100,000 books in the entire system.

This is a polynomial-time operation because $O(1)$ and $O(\log r)$ are both polynomial. In the even of an optimistically pessimistic approach, even in the case of a worst-case scenario for database indexes in which a B-tree search of 100,000 items takes at worst $O(\log n)$ steps, $\log_2(100,000) = 16.6 < 17$ steps, well under the limit for a two-second maximum response time.

3.2.2 Operation 2: Dynamic Connectivity Update

The Dynamic Connectivity procedure occurs whenever there is a new borrow event that provides evidence that the books are related. The process would involve the comparison of the newly borrowed book with the average student history of 30 books. For each book from this average student history, Find can be applied on both books, after which the system may decide to perform the Union operation if their representatives are different. The amortized running time of each of these operations in the optimized union-find algorithm is $O(\alpha(n))$, and such an asymptotic is discussed recently by researchers analyzing the Union-Find operation (Huang et al., 2023).

Given that h denotes the average student history length, the approximate running time of one borrowing update will therefore be $O(h \alpha(n))$. When considering $h = 30$ and $n = 100,000$, this will amount to almost nothing. Assuming that the number of borrow events in one day is 500, the system would have to do about 15,000 relationship checks before using the similarity criterion.

Operation	Best case	Average / amortized case	Worst case note	Suitability
Find with Path Compression	$O(1)$ when the node already points to the root.	$O(\alpha(n))$ amortized with union by size/rank.	A single early call may traverse a taller tree before compression.	Excellent for repeated real-time checks.
Union	$O(1)$ if both books are already in the same component after Find.	$O(\alpha(n))$ amortized because it performs two Find calls.	Without balancing and compression, trees could become inefficient.	Suitable for every borrowing update.
Connectivity Query	$O(1)$ when both representatives are cached or already compressed.	$O(\alpha(n))$ amortized for two Find calls.	Worst theoretical cost depends on implementation quality.	Suitable for recommendation candidate filtering.
Availability Check	$O(1)$ average hash lookup.	$O(1)$ lookup plus $O(\log r)$ update if return queue is used.	Poor hashing can degrade, so production indexing is required.	Suitable for two-second response target.

Table 3: Best-case and worst-case considerations for selected algorithms

3.3 Practical Implication for Real-Time Smart Library

P problems are relevant to Smart Library as their processing time changes in a predictable manner in relation to the increase in the input size. According to the requirements provided in the assignment prompt, the search should take no more than two seconds, and thousands of requests should be served per day (Asia Pacific University of Technology and Innovation, 2026). Operations like availability checking and dynamic DSU connectivity checks may be arranged to affect only a part of the data set rather than access the entire database.

For instance, availability check will only involve one book record and, possibly, one return-date queue. Dynamic DSU operation will involve just the new book added to the DSU and some 30 other books previously borrowed by a particular student rather than all 100,000 books stored in the database. Local processing becomes very valuable especially when the system has over 500 users at its peak and can divide many polynomial operations among database indexes and application server nodes. Recommendation optimization, on the contrary, is not feasible for the online solution due to multiple iterations.

4.0 Question 2: Understanding Non-Deterministic Polynomial-Time (NP) Problem

4.1 Concept and Identification

NP-problem definition states that it is a type of decision problem that can be checked within polynomial time using a deterministic verifier or a nondeterministic machine in polynomial time (Universitat Politecnica de Catalunya, 2023). Computational-complexity notes from the recent studies define NP as $\text{NTIME}(\text{poly}(n))$. The problem is usually hard to find an exact solution, but there exists an efficient way of checking its correctness.

The Smart Library operation, which is classified as an NP problem or NP-hard task, within this assignment, is fully optimal personalization recommendations. As it was mentioned above, the specification for the recommendation engine claims that it needs to suggest 10 similar books for each student based on their borrowing records. It is relatively easy to provide the top-10 ranking in a polynomial way. However, if one tries to provide the fully optimal recommendation, which means selecting the best combination of 10 books based on multiple criteria such as student similarity, subject diversity, book availability, popularity, and novelty, the complexity will increase drastically. Metaheuristic frameworks have been widely applied in recommenders-systems literature to limit decision-making search space when facing non-linear NP-hard problems (Bajenaru et al., 2023).

4.2 Complexity Analysis of Fully Optimal Recommendation

If c is the number of candidate books after filtration, then the concrete problem could be represented as selection of 10 books from c options. Thus, we have the number of different possible recommendation sets equal to $C(c, 10)$, which can be defined as $c!/10!(c - 10)!$. In case of scoring all possible recommendation sets and calculation of diversity between 10 chosen books, the algorithm will face approximately $O(C(c, 10) \cdot 10^2)$. This issue is combinatorial and will grow extremely fast as c grows.

If $c = 200$ is already too big for an online computation, then $C(100,000, 10)$ is absolutely unacceptable if there are around 100,000 books in the whole database. In case of growing borrowing history, the problem gets even more complicated because now each candidate should be compared with many past books. In case of h books in borrower's history, calculating the candidate similarity score can be done in time $O(h \cdot f)$, where f is the number of features. In this case, the overall estimation of exact search can be written as $O(C(c, 10) \cdot 10 \cdot h \cdot f)$.

Recommendation approach	Candidate search space	Complexity trend	Real-time suitability
Exact fully optimal set of 10	All 10-book combinations from c candidates.	Combinatorial: $O(C(c, 10) * \text{scoring cost})$.	Not suitable for live requests.
DSU-filtered greedy ranking	Books inside same or nearby components.	Polynomial: $O(c \log c)$ or $O(c * h * f)$ after filtering.	Suitable if c is limited and cached.
Cached precomputed recommendations	Previously computed top candidates per component/student profile.	$O(1)$ to $O(k)$ retrieval, plus background refresh.	Highly suitable for repeated requests.

Table 4: Exact recommendation compared with polynomial alternatives

4.3 Why the NP approach becomes Inefficient

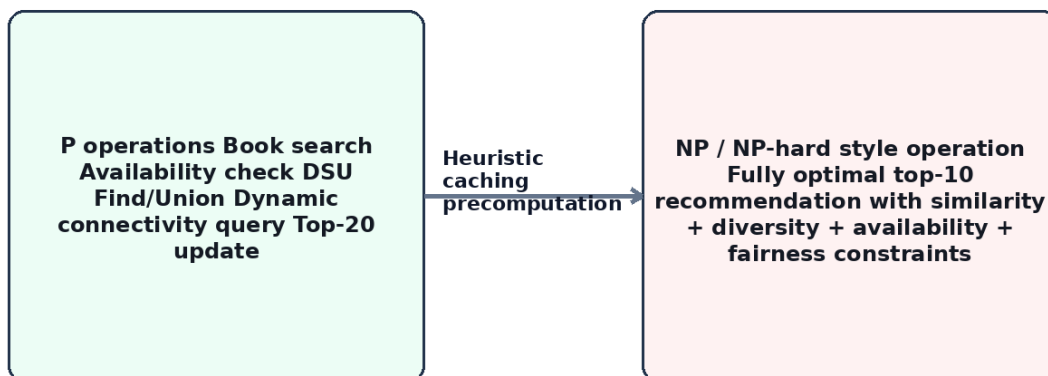
The inefficiency of the NP-style approach lies in two areas. Firstly, the amount of books will influence the candidate pool. The reason is that Smart Library owns 100,000 books, and even a marginal increase in the size of the candidates can lead to a considerable explosion of 10-books combinations. Secondly, the history will increase the scoring cost. The requirement from the assignment specifies that on average each student has a history of 30 books, but this value may be much higher in case of frequent users.

It is unsuitable to do in real-time since the system faces an average number of 300 recommendation requests per day, which sometimes face peaks in demand at certain hours of the day. A single request trying to enumerate all combinations will occupy CPU and RAM that should be utilized for searching and borrowing operations. Therefore, we need to develop a scalable solution by translating our exact optimization problem to heuristic approximation.

5.0 Question 3: P vs NP Comparison in Smart Library

The fundamental distinction between P and NP in Smart Library is the distinction between operations that can be performed reliably while using the library normally and operations that might require an impractical search among numerous possibilities to find the right answer. Operations that belong to P have limited polynomial time and can be designed to achieve performance goals. Even though NP or NP-hard optimization problems can be helpful in modeling the best case scenario, solving the problem precisely will not be done on-line.

P vs NP Placement in Smart Library



Goal: keep daily interactive work in low-degree polynomial time Goal: avoid exact combinatorial search during real-t

Figure 5: P vs NP placement of Smart Library operations

Comparison factor	P operations in Smart Library	NP / NP-hard style recommendation optimization
Time complexity	$O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, or $O(\alpha(n))$ for DSU operations.	Exponential or combinatorial, such as $O(C(c, 10) * \text{scoring cost})$.
Execution time	Predictable and suitable for online requests.	Unpredictable when candidate set grows; may exceed response-time target.
Scalability	Scales well to 100,000 books when indexes, caches, and DSU arrays are used.	Scales poorly because each new candidate increases the search space sharply.
Examples	Availability check, book search, top-20 update, DSU Find/Union, dynamic connectivity query.	Fully optimal top-10 recommendation with similarity, diversity, availability, and fairness constraints.
System role	Must remain online and real-time.	Can be approximated online or solved partially in background batches.

Table 5: P vs NP comparison for the Smart Library case study

5.1 Which Operation must Remain in P and Why

Book search, book availability, borrowing, book return, popular book identification, and dynamic connectivity update must stay in class P. Those operations are related directly to the user experience of the library services provided to the student. Otherwise, they would make the requirement of two seconds' response time infeasible. Book search can be done through an inverted index and exact hash ISBN match. Availability through direct indexing by book ID and trivial stock and loan database. Popular books identification should include counter table and top-20 heap or cache. DSU update must rely on path compression in Find and Union by rank or size to keep similar components efficient.

Dynamic connectivity update plays a special role here because it makes possible reduction in the computational complexity of finding recommendations. For example, before comparing the student's profile with all 100,000 books, the system may find only connected components including books borrowed by the student. The online query will have a reduced list of candidates to rank and evaluate. This way, the actual recommendations remain polynomial despite the NP-hard properties of optimal problem solving.

5.2 Which Operations can Tolerate Higher Complexity

Operations which can support a greater degree of complexity include offline analysis, nightly recommendations, library acquisitions, and pattern mining over a long period of time. None of these functions requires immediate response within two seconds. It may be possible to re-calculate the similarity components from the latest borrowing data overnight, improve the recommendation scores for all components, and prepare them for use during the following day. Nonetheless, the program must avoid using exact exponential solutions based on the entire library database.

6.0 Question 4: Decision Making and Optimization

6.1 Algorithm Selection

As the designer of the system, my choice of solution would be an approximate or heuristic recommender instead of the optimal NP-based solution. The reason for this decision lies in the nature of the Smart Library – it is an interactive system that requires two seconds for a response time, and which has 300 recommendation requests per day, 2,000 searches per day, and hundreds of accesses during peak hours (Asia Pacific University of Technology and Innovation, 2026). While the optimal solution can find the better set mathematically, its performance can make the solution unacceptable.

The proposed design uses a hybrid pipeline. First, DSU will manage the dynamic similarity components. Secondly, we will use Find with Path Compression for finding the relevant components based on the student's history. Thirdly, the candidates will be collected from these components. Fourthly, the solution will use the greedy score that considers similarities, availability, and popularity, among others. Fifthly, the solution will be cached for future uses. This solution compromises global optimality of the top-10 recommendations.

6.2 Real-World Optimization Technique: Precomputation and Caching with Greedy Ranking

One way to tackle NP complexity involves the use of precomputation, caching, and greedy ranking. By precomputing, we mean that costly similarity scores and component statistics are computed before the student makes his/her request. Caching entails the storing of results for the best candidate components or student profiles. This is done in order to avoid computing the same result several times. Greedy ranking entails the selection of the next book one after another based on a certain scoring algorithm rather than listing all the combinations of 10 books.

The scoring function may take the form: $\text{Score}(\text{book}) = 0.45(s) + 0.20(a) + 0.15(p) + 0.10(n) + 0.10(d)$ where "s" stands for similarity and so on. The exact values of the weights may be adjusted by the library personnel. Diversity bonus may be used to penalize books that bear close resemblance to those previously selected. Greedy ranking does not guarantee global optimality, but it is simple, clear, fast, and easy to debug. It is more suited for our case than an NP approach.

Technique	How it works	Performance improvement	Trade-off
DSU candidate filtering	Use Find to locate related book components from student history.	Reduces candidate set from 100,000 books to a smaller component-based set.	May miss useful books outside the component.
Precomputation	Calculate similarity and component statistics before live requests.	Moves heavier work away from the online response path.	Recommendations may be slightly stale until refresh.
Caching	Store top candidates for popular components and active students.	Repeated recommendation requests become near $O(1)$ retrieval.	Cache invalidation is required after major updates.
Greedy ranking	Select top books one by one using a weighted score.	Avoids $O(C(c, 10))$ exact enumeration.	Does not guarantee global optimality.

Table 6: Techniques to handle NP complexity in Smart Library recommendations

6.3 Proposed Online Recommendation Algorithm

```
17 Recommend(studentID):
18     candidateSet = empty set
19     for book in BorrowHistory[studentID]:
20         root = Find(book)
21         candidateSet.add(TopCandidatesByComponent[root])
22
23     candidateSet.remove(books already borrowed by studentID)
24     candidateSet.remove(books currently unavailable beyond acceptable return window)
25
26     selected = empty list
27     while len(selected) < 10 and candidateSet is not empty:
28         best = argmax Score(book, selected, studentID) over candidateSet
29         selected.add(best)
30         candidateSet.remove(best)
31
32     CacheRecommendation(studentID, selected)
33     return selected
34
```

Figure 6: Pseudocode for the recommended polynomial online recommendation pipeline

The complexity of this online pipeline is polynomial. If there are c books in the filtered set of candidates, then the ranking can be done as either $O(c \log c)$, if based on sorting, or as $O(10c)$, since we repeatedly choose 10 books from those candidates. The former is better than the latter, which in turn is much better than $O(C(c, 10))$. This immediately proves the required efficiency of the system.

7.0 Conclusion

It can be clearly seen in the case study of Smart Library that computational complexity plays a critical role in practical system development since the operations mentioned above including availability, book search, update of popular books, and dynamic connectivity using DSU need to stay in polynomial time, since they have to be performed while executing the actual live application. The selected DSU algorithm based on Find with Path Compression and Dynamic Connectivity will allow the recommendation module to start working with the initial candidate set of size $k = O(1)$.

However, solving the fully optimal recommendation problem seems to be impossible due to its inherent nature – finding the optimal combination of the 10 books among all candidates while taking into account similarity, diversity, availability, and fairness simultaneously would create combinatorially complex task unsuitable for live system execution. Thus, the most suitable approach for the problem is not to find the optimal solution of the problem but rather to filter the initial candidate set using DSU and rank the rest.

8.0 References

1. Asia Pacific University of Technology and Innovation. (2026). CT065-3-3 Algorithmics Assignment Question - Part 2 [Assignment brief].
2. Bajenaru, V., Lavoie, S., Benyo, B., Riker, C., Colby, M., & Vaccaro, J. (2023). Recommender system metaheuristic for optimizing decision-making computation. *Electronics*, 12(12), 2661. <https://doi.org/10.3390/electronics12122661>
3. Bun, M. (2023). Lecture notes 2: Time complexity, P, NP, NP-completeness. Boston University. https://cs-people.bu.edu/mbun/courses/535_F23/lectures/lec2.pdf
4. Huang, Z., Lambert, C., Nie, Z., & Peng, R. (2023). Simpler analyses of union-find. arXiv. <https://arxiv.org/abs/2308.09021>
5. The Linux Kernel documentation. (2024). Union-Find in Linux. https://docs.kernel.org/core-api/union_find.html
6. Universitat Politècnica de Catalunya. (2023). Computational complexity: Classes. <https://www.cs.upc.edu/~mjserna/docencia/gm-aic/2023/2-AiC-Comp.pdf>

Appendix A: AI Tool Disclosure

Use of the AI tool was to help plan, draft, format, and perform quality checks on this assignment document. It was used to interpret the assignment description, organize answers based on Questions 1 to 4, and modify the selected data structures and algorithms, i.e., Disjoint Set Union, Find with Path Compression, and Dynamic Connectivity, in relation to the Smart Library case study. The tool was appropriate to be used here due to the need to provide a technical response and write in an academic style in terms of complexity analysis, Big-O notation, comparison of P and NP, and systems design.

The output from the AI tool was not taken at face value. The content generated by the tool was carefully checked for relevance to the case study, compliance with questions' requirements, and appropriateness with respect to the algorithms chosen. The answer given was changed appropriately to remove any unfounded statements, make usage of DSU more realistic, and recognize some of the limitations of DSU, which do not include ability to delete the connectivity edges in a normal implementation. Sources cited were restricted to those published in or after 2023.